

Roteiro de Aula - Busca Linear e Busca Binária

1. Definição e Fundamentação Teórica

Busca Linear (ou Sequencial)

A Busca Linear é o algoritmo de busca mais simples. Ela percorre o vetor elemento por elemento, a partir da primeira posição ($i = 0$), comparando o valor desejado x com cada elemento do vetor utilizando o método `equals`.

- **Características:** Funciona em vetores ordenados ou não ordenados.
- Utiliza exclusivamente a verificação de igualdade lógica com `equals` (`x.equals(V[i])`). Não necessita da interface `Comparable`.
- No pior caso (elemento no final ou ausente), realiza N comparações.
- Complexidade temporal de pior caso: $O(n)$.

Busca Binária

A Busca Binária é um algoritmo altamente eficiente baseado no paradigma de Divisão e Conquista. O funcionamento é semelhante à busca de uma palavra em um dicionário impresso: abrimos no meio; se a palavra procurada for menor, continuamos a busca na primeira metade; se for maior, na segunda metade.

- **Exigência fundamental:** O vetor DEVE estar previamente ordenado.
- Necessita obrigatoriamente da interface `Comparable` (método `compareTo`) ou de um `Comparator` (método `compare`) para decidir a direção (esquerda ou direita).
- A cada passo, calcula-se o índice central $m = (\text{inicio} + \text{fim}) / 2$.
- Se $x == V[m]$, o elemento foi encontrado (sucesso).
- Se $x < V[m]$, reduz para a metade esquerda ($\text{fim} = m - 1$).
- Se $x > V[m]$, reduz para a metade direita ($\text{inicio} = m + 1$).
- Complexidade temporal de pior caso: $O(\log_2 n)$.

Diferença entre `equals` e `compareTo` / `Comparator` na Prática

- **`equals` (Usado na Busca Linear):** Verifica se dois objetos são logicamente iguais (ex.: duas pessoas com mesmo CPF). Retorna apenas `true` ou `false`. Como a Busca Linear não precisa saber a ordem, o `equals` é suficiente.
- **`compareTo` / `Comparator` (Usado na Busca Binária):** Determina a relação de ordem entre objetos (retorna negativo se menor, zero se igual, e positivo se maior). A Busca Binária precisa obrigatoriamente desse retorno relativo para decidir a direção da busca.

2. Antes de programar: simule uma execução

Simulação 1: Busca Linear

Considere o vetor $V = [14, 27, 39, 46, 55, 63, 71, 80, 92]$. Simule o algoritmo para localizar $x = 63$ (utilizando a comparação com equals) e complete a tabela abaixo:

Passo	Índice i	Elemento V[i]	Comparação (x.equals(V[i]))	Resultado / Próxima Ação
1	0	14	63.equals(14) -> Falso	Avança para i = 1
2	1	27	63.equals(27) -> Falso	Avança para i = 2
3	i = 2	39	63.equals(39) -> Falso	Avança para i = 3
4	i = 3	46	63.equals(46) -> Falso	Avança para i = 4
5	i = 4	55	63.equals(55) -> Falso	Avança para i = 5
6	i = 5	63	63.equals(63) -> Verdadeiro	Encontrou

Pergunta: Na Busca Linear, por que podemos usar o método equals em vez de compareTo?

A Busca Linear apenas verifica se o elemento procurado é igual ao elemento atual. Ela não precisa determinar uma ordem entre os elementos.

Simulação 2: Busca Binária

Considere o mesmo vetor ordenado $V = [14, 27, 39, 46, 55, 63, 71, 80, 92]$ (tamanho $N = 9$, índices 0 a 8).

Caso A: Pesquisa pelo elemento $x = 63$ (Busca com Sucesso)

Passo	Início (i)	Fim (f)	Meio (m)	Elemento V[m]	Comparação (compare(x, V[m]))	Próximo Intervalo
1	0	8	4	55	63 > 55 (Maior -> Direita)	i = 5, f = 8
2	5	8	6	71	63 < 71 (Menor -> Esquerda)	i = 5, f = 5
3	—	—	—	—	—	—

Caso B: Pesquisa pelo elemento $x = 50$ (Busca com Fracasso)

Passo	Início (i)	Fim (f)	Meio (m)	Elemento V[m]	Comparação (compare(x, V[m]))	Próximo Intervalo
1	0	8	4	55	50 < 55 (Menor -> Esquerda)	i = 0, f = 3
2	0	3	—	—	—	—
3	—	—	—	—	—	—
4	—	—	—	—	—	—

5

i > f (Intervalo
Vazio!)

Pergunta: Como a Busca Binária usa o resultado da comparação para decidir entre a metade esquerda e a metade direita?

Se `compare(x, V[m])` retornar um valor negativo, significa que `x` é menor que `V[m]`, então a busca continua na metade esquerda. Se retornar um valor positivo, `x` é maior que `V[m]`, então a busca continua na metade direita. Se retornar zero, o elemento foi encontrado.

Pergunta: Quantos itens são examinados no máximo pela Busca Binária num vetor de 5.000 elementos?

Seria basicamente quantas vezes ele pode ser dividido por 2

5000 daria $12,29 + 1 = 13$

logo 13 comparações no máximo

pior caso $\log(n) + 1$

Pergunta: Para um vetor de 131.000 posições, qual é o número máximo de comparações na Busca Binária?

Podemos pegar quantas vezes o 2 elevado que dá um número aproximado a 131000. Nesse caso seria 2 elevado a 17 que daria 131072

Nesse caso 17 comparações no máximo

3. Complete o código da classe Busca

Crie a classe genérica `Busca<T>`. Observe que a Busca Linear utiliza diretamente o método `equals`, permitindo buscar em qualquer tipo de objeto, enquanto a Busca Binária utiliza `Comparator` / `Comparable`. Complete as lacunas do código (_____).

```
import java.util.Comparator;

public class Busca<T> {
    private long contaComparacoes;

    public long getContaComparacoes() {
        return contaComparacoes;
    }

    // Busca Linear utilizando o método equals
```

```
public int linear(T[] array, T x) {
    this.contaComparacoes = 0;
    for (int i = 0; i < array.length; i++) {
        // contar comparações
        contaComparacoes++;
        if (x.equals(array[i])) {
            return i;
        }
    }
    return -1;
}

// Busca Binária
public int binaria(T[] array, T x, Comparator<T> comparator) {
    this.contaComparacoes = 0;
    int inicio = 0;
    int fim = array.length - 1;

    while (inicio <= fim) {
        int meio = (inicio + fim) / 2;
        // contar comparações
        contaComparacoes++;
        int comparacao = comparator.compare(x, array[meio] );

        if (comparacao == 0) {
            return meio;
        }
        if (comparacao < 0) {
            fim = meio - 1;
        } else {
            inicio = meio + 1;
        }
    }
    return -1;
}

// Sobrecarga da Busca Binária utilizando (Comparable)
public int binaria(T[] array, T x) {
    return binaria(array, x, (a, b) -> ((Comparable<T>) a).compareTo(b));
}
}
```

Pergunta: Por que o método linear necessita apenas do método equals, enquanto a binaria precisa de Comparator ou Comparable?

A Busca Linear necessita apenas de equals porque verifica somente a igualdade entre os elementos. Já a Busca Binária precisa de Comparator ou Comparable para determinar se o elemento procurado é menor, igual ou maior que o elemento do meio e, assim, escolher qual metade do vetor pesquisar.

4. Implementando o main

Agora crie uma classe App (ou atualize a existente no projeto JavaSorts) para executar a Busca Linear e a Busca Binária no mesmo vetor inicial e comparar o tempo e quantidade de comparações.

Atenção: Lembre-se: A Busca Linear funciona em qualquer vetor, mas para a Busca Binária funcionar, o vetor DEVE estar ordenado!

```
public class App {
    public static Integer[] gerarVetorAleatorio(int tamanho) {
        Integer[] vetor = new Integer[tamanho];
        for (int i = 0; i < tamanho; i++) {
            vetor[i] = (int) (Math.random() * 1000000);
        }
        return vetor;
    }

    public static void main(String[] args) throws Exception {
        int n = 50000;
        long inicio, fim;

        System.out.println("=== DEMONSTRACAO DE BUSCA (LINEAR E BINARIA) ===");
        Integer[] vetorBusca = gerarVetorAleatorio(n);
        // apresente o restante do código implementado aqui
    }
}
```

5. Comparando resultados

Execute o programa e preencha a tabela com os valores exibidos no console:

Algoritmo	Critério Utilizado	Elemento Pesquisado	N	Comparações	Tempo (ms)
Busca Linear	equals	Presente (N/2)	50.000		
Busca Binária	compareTo	Presente (N/2)	50.000		
Busca Linear	equals	Ausente (Pior Caso)	50.000		

ESTRUTURA DE DADOS II

PROF. MATHEUS FRANCO

Busca Binária	compareTo	Ausente (Pior Caso)	50.000
Busca Linear	equals	Presente (N/2)	150.000
Busca Binária	compareTo	Presente (N/2)	150.000
Busca Linear	equals	Ausente (Pior Caso)	150.000
Busca Binária	compareTo	Ausente (Pior Caso)	150.000
Busca Linear	equals	Ausente (Pior Caso)	1.000.000
Busca Binária	compareTo	Ausente (Pior Caso)	1.000.000

Fique ligado: premissas e conclusões da aula!

- A Busca Linear baseia-se em equals e realiza N comparações no pior caso.
- A Busca Binária baseia-se em compareTo/Comparator e realiza no máximo $\log_2 N + 1$ comparações no pior caso.
- O tempo de execução da Busca Binária é praticamente instantâneo (0 ms), enquanto a Busca Linear cresce proporcionalmente ao tamanho de N.

Pergunta: Na classe Pessoa, se desejarmos buscar um aluno por CPF (igualdade lógica única), o método equals na Busca Linear resolve o problema? E se quisermos usar Busca Binária por Nome, que método/interface é obrigatório implementar?

Sim. Na Busca Linear, o método equals resolve a busca por CPF, desde que a igualdade da classe Pessoa seja definida pelo CPF. Já para realizar uma Busca Binária por nome, é necessário utilizar Comparable (compareTo) ou um Comparator que compare os nomes. Além disso, o vetor deve estar previamente ordenado por nome.